Práctica 1: Extracción y Análisis de Datos Cinemáticos del Robot UR3e con ROS 2

Contenido

1. **Objetivo de la Práctica** **2**
2. **Entorno de Trabajo (Punto de Partida)** **2**
   a. ¿Cómo arrancar el entorno? **2**
3. **Tareas** **3**
   Parte 1: Requisitos Mínimos 3
   Parte 2: Retos Opcionales 3
   Parte 3: Calidad del Código 3
4. **Evaluación y Detalles** **5**
5. **Criterios de Calificación:** **5**
6. **Penalizaciones:** **5**
7. **Entrega** **6**
8. **Recursos de Apoyo y Notas Extra** **6**

1. Objetivo de la Práctica

El objetivo de esta práctica es familiarizarse con la extracción y procesamiento de datos en entornos robóticos. Trabajaréis con un entorno simulado de un brazo robótico **UR3e** utilizando **ROS 2 Jazzy**.

Vuestra misión principal es desarrollar un nodo (programa) en **Python** que se suscriba al *topic* que publica el estado de las articulaciones del robot. A partir de la lectura de estos datos durante el movimiento del robot, deberéis extraer la evolución temporal (respecto al tiempo) de la **posición, velocidad y aceleración**.

2. Entorno de Trabajo (Punto de Partida)

Partiendo del entorno Dockerizado de ROS2 y **Visual Studio Code**. En el entorno se incluye:

- La simulación completa del robot UR3e.
- Todos los paquetes de ROS 2 necesarios.
- Scripts de ejemplo que ya contienen la lógica básica para mover el robot y para publicar/suscribirse a un *topic*.

Deberéis generar un paquete propio en ROS 2 Jazzy donde alojaréis todo el contenido desarrollado en la práctica.

a. ¿Cómo arrancar el entorno?

En el archivo "README.md" del repositorio base encontraréis los detalles y requisitos del entorno. Una vez cumplidos requisitos mínimos basta con ejecutar el script proporcionado según vuestro sistema operativo:

- Ejecutar "./lab.sh iniciar" para **Linux** o "lab.bat iniciar" para **Windows** (incluir "--lab" o "--lab-gpu" si se quiere lanzar una de estas opciones)

3. Tareas

Parte 1: Requisitos Mínimos

Para superar la práctica, debéis desarrollar un script en Python que cumpla exactamente con lo siguiente:

1. **Crear un Suscriptor en ROS 2** que lea en tiempo real el estado de las articulaciones del robot.
2. **Ejecución de múltiples trayectorias:** El robot deberá ejecutar hasta 3 trayectorias diferentes, las trayectorias deben incluir movimientos Lineales, Joint y Circulares para la ingesta de datos.
3. **Procesar los datos:** Extraer y almacenar la posición, velocidad y aceleración de las articulaciones vinculadas a su marca de tiempo (tiempo de simulación o de reloj) de forma diferenciada para cada una de las trayectorias ejecutadas.

Parte 2: Retos Opcionales

Una vez conseguido el objetivo principal, podéis implementar estas características para optar a una mayor calificación:

- **Mensaje ROS 2 personalizado:** Crear un archivo de tipo "msg" de ROS 2 personalizado que empaquete y envíe simultáneamente los datos calculados de posición, velocidad y aceleración en un solo mensaje.
- **Ingesta inteligente de datos:** El suscriptor debe procesar y guardar datos de forma automática **únicamente cuando el robot se esté moviendo**, ignorando por completo los periodos en los que se encuentra estático.

Parte 3: Calidad del Código

Se evaluará que el código entregado sea propio de un entorno "profesional". Se exige:

- Código limpio y bien estructurado. Programación orientada a objetos.
- Uso de comentarios explicativos (no triviales) y Docstrings en funciones y clases.
- Uso de Type Annotations / Type Hinting (indicación de los tipos de variables esperados).

- Seguir el estándar de nomenclatura de Python (PEP-8):
  - `variables\_y\_funciones` en minúsculas y separadas por barra baja (mi_función, variable_a).

- `NombresDeClases` con la primera letra en mayúscula y sin espacios ni guiones (MiClase).
- "Funciones privadas de la clase" con barra baja delante del nombre (_function)
- El nombre de variables, clases y funciones debe ser lógico, y entendible, sin variables ni valores mágicos.

4. Evaluación y Detalles

Esta es una **práctica en parejas**, pero **la evaluación del trabajo será estrictamente individual** basándose en la actividad del repositorio.

1. Sobre el uso de GitHub:

El código debe alojarse obligatoriamente en un repositorio de **GitHub**.

- **El trabajo de cada miembro del grupo será evaluado ÚNICAMENTE a través de sus *commits*.** Si un alumno no tiene *commits* a su nombre en el repositorio, su nota será un 0, independientemente del trabajo de su pareja. Debéis trabajar de forma colaborativa usando Git.
- No puede haber ningún contribuidor externo al grupo en el repositorio.

5. Criterios de Calificación:

- **50%:** Cumplimiento de los requisitos mínimos.
- **40%:** Implementación de las funcionalidades opcionales.
- **10%:** Limpieza de código y buenas prácticas.

6. Penalizaciones:

- **Fuera de plazo:** Una entrega fuera de plazo supone una nota máxima del 75% si se entrega en un plazo menor a 24h posteriores de la finalización del plazo original. Cualquier *commit* registrado en el repositorio con una fecha y hora posterior a la fecha límite de entrega se considerará como una entrega tardía, en caso de entrega fuera del plazo de 24 horas extra, la tarea será evaluada como un 0.
- **Uso abusivo de IA:** El uso de herramientas de Inteligencia Artificial de forma abusiva o copiando bloques de código sin comprender su funcionamiento supondrá un 0 inmediato. El profesor podrá preguntar al alumno en caso de duda.

7. Entrega

Qué entregar: Únicamente el enlace (URL) a vuestro repositorio de GitHub.

- Aseguraos de que el repositorio sea accesible para el equipo docente el día de la entrega. El nombre del repositorio es libre.

8. Recursos de Apoyo y Notas Extra

Para facilitar el desarrollo y fomentar las buenas prácticas de ingeniería, os recomendamos apoyaros en las siguientes herramientas durante la práctica:

- **Herramienta rqt_graph:** Se trata de una interfaz visual de ROS 2 que os permitirá ver un esquema gráfico de los nodos que están activos y cómo se comunican entre sí a través de los topics. Es una herramienta para depurar errores, entender el flujo de los datos y aseguraros de que vuestro suscriptor está correctamente conectado al topic del robot. Podéis ejecutarla mediante el paquete de ROS 2 "rqt_graph".
- **Documentación Oficial de ROS 2:** Acostumbraos a consultar la documentación y los tutoriales oficiales de ROS 2 (específicamente para la distribución Jazzy). Es la fuente más fiable para entender el funcionamiento de los suscriptores, la creación de mensajes personalizados y la estructura de los paquetes. Encima está bien escrita ;).

Sabemos que ROS 2 y Docker pueden tener una curva de aprendizaje inicial compleja. Si os encontráis con un error que os impide avanzar y habéis intentado solucionarlo sin éxito apoyándoos tanto en la documentación oficial como mediante preguntas a la IA, **no dudéis en preguntar al profesor**. Estamos aquí para ayudaros a entender los conceptos y guiaros en la resolución de problemas. No esperéis al último día para preguntar, es posible que no podamos responder en plazos tan cortos de tiempo y no se harán excepciones.